

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN GRADUATION RESEARCH 2

Phát hiện tấn công mạng bằng phát hiện bất thường dựa trên phân tích dữ liệu

ĐỖ TUẤN MINH

minh.dt225741@sis.hust.edu.vn

Ngành Công nghệ thông tin

Chuyên ngành Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: PGS. TS. Nguyễn Linh Giang _____
Viện: SoIct, Đại học Bách Khoa Hà Nội

HÀ NỘI, 7/2025

Nội dung

TÓM TẮT(ABSTRACT).....	4
1. GIỚI THIỆU VÀ ĐẶT VẤN ĐỀ.....	4
1.1. Bối cảnh an ninh mạng.....	4
1.2. Từ phân loại nhị phân đến phân loại đa lớp.....	5
1.3. Mục tiêu nghiên cứu.....	5
2. PHÂN TÍCH TẬP DỮ LIỆU NSL-KDD.....	6
2.1. Tổng quan về tập dữ liệu.....	6
2.2. Phân tích chi tiết 5 nhóm lớp hành vi	6
2.3. Vấn đề mất cân bằng dữ liệu nghiêm trọng	7
2.4. Phân tích đặc trưng.....	8
3. PHƯƠNG PHÁP NGHIÊN CỨU VÀ TIỀN XỬ LÝ.....	10
3.1. Quy trình tiền xử lý	10
3.2. Chiến lược xử lý mất cân bằng (Imbalance Handling)	12
3.3. Các mô hình và cấu hình thực nghiệm.....	13
3.3.1. K-Nearest Neighbors (KNN)	13
3.3.2. Random Forest (RF).....	14
3.3.3. XGBoost phân tầng (Hierarchical XGBoost)	14
3.3.4. Long Short-Term Memory (LSTM).....	15
3.3.5. Deep Support Vector Data Description (Deep SVDD).....	16
3.3.6. Variational Autoencoder (VAE)	17
3.3.7. Transformer	18
4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	20
4.1. Tiêu chí đánh giá (Evaluation Metrics).....	20
4.2. Kết quả mô hình K-Nearest Neighbors (KNN).....	21
4.3. Kết quả mô hình Random Forest (RF).....	21
4.4. Kết quả mô hình XGBoost (Chiến lược phân tầng).....	22
4.5. Kết quả mô hình LSTM	23
4.6. Kết quả mô hình Deep SVDD.....	24
4.7. Kết quả mô hình VAE	24
4.8. Kết quả mô hình Transformer	25
5. THẢO LUẬN VÀ SO SÁNH.....	26
5.1. Tổng hợp kết quả và so sánh mô hình.....	26

5.2.	Phân tích sự thất bại trong nhận diện lớp hiếm.....	27
5.3.	So sánh với hiệu quả của bài toán nhị phân	28
5.4.	Phân tích ưu nhược điểm của từng phương pháp.....	28
6.	KẾT LUẬN.....	29

TÓM TẮT(ABSTRACT)

Trong bối cảnh các cuộc tấn công mạng ngày càng tinh vi và đa dạng, việc phát hiện xâm nhập không chỉ dừng lại ở việc cảnh báo "có tấn công" (phân loại nhị phân) mà còn đòi hỏi khả năng định danh cụ thể loại hình tấn công (phân loại đa lớp) để đưa ra phản ứng phù hợp và kịp thời. Báo cáo này trình bày kết quả nghiên cứu thực nghiệm toàn diện việc áp dụng cả các thuật toán học máy truyền thống và các kiến trúc học sâu tiên tiến cho bài toán phân loại 5 lớp hành vi mạng trên tập dữ liệu chuẩn NSL-KDD.

Nghiên cứu triển khai và đánh giá 7 mô hình khác nhau: **K-Nearest Neighbors (KNN)**, **Random Forest (RF)**, **XGBoost phân tầng (Hierarchical XGBoost)**, **Long Short-Term Memory (LSTM)**, **Deep Support Vector Data Description (Deep SVDD)**, **Variational Autoencoder (VAE)**, và **Transformer**. Đặc biệt, nghiên cứu tập trung giải quyết thách thức lớn nhất của tập dữ liệu này: sự mất cân bằng nghiêm trọng giữa các lớp, đặc biệt là sự khan hiếm của các mẫu tấn công R2L (Remote to Local) với 0.79% và U2R (User to Root) với chỉ 0.04%.

Kết quả thực nghiệm cho thấy các mô hình học sâu như **Transformer** và **LSTM** đạt được hiệu suất cao nhất với Accuracy lần lượt là **78.2%** và **76.5%**, vượt trội so với các phương pháp học máy truyền thống. Tuy nhiên, việc nhận diện các lớp hiếm (R2L, U2R) vẫn còn là thách thức đáng kể. Báo cáo đi sâu phân tích nguyên nhân của sự khác biệt hiệu năng giữa các mô hình và đề xuất các hướng đi tương lai để cải thiện khả năng phát hiện các loại tấn công hiếm.

1. GIỚI THIỆU VÀ ĐẶT VẤN ĐỀ

1.1. Bối cảnh an ninh mạng

Sự bùng nổ của Internet, điện toán đám mây và Internet vạn vật (IoT) đã mở ra kỷ nguyên kết nối toàn cầu chưa từng có. Tuy nhiên, đi kèm với sự phát triển này là sự gia tăng đáng lo ngại về tần suất, quy mô và mức độ tinh vi của các cuộc tấn công mạng. Theo báo cáo của các tổ chức an ninh mạng quốc tế, số lượng các cuộc tấn công mạng đã tăng hơn 300% trong giai đoạn 2020-2024, với thiệt hại ước tính lên tới hàng nghìn tỷ đô la mỗi năm.

Các hệ thống phát hiện xâm nhập (Intrusion Detection System - IDS) truyền thống chủ yếu dựa trên phương pháp so khớp chữ ký (signature-based). Phương pháp này hoạt động bằng cách so sánh lưu lượng mạng với cơ sở dữ liệu các mẫu tấn công đã biết. Mặc dù hiệu quả với các tấn công đã được ghi nhận, phương pháp này bộc lộ điểm yếu nghiêm trọng:

- **Không phát hiện được tấn công mới (zero-day attacks):** Các tấn công chưa có chữ ký trong cơ sở dữ liệu sẽ hoàn toàn lọt qua hệ thống phòng thủ.
- **Khó theo kịp các biến thể:** Kẻ tấn công có thể dễ dàng thay đổi nhỏ trong mẫu tấn công để vượt qua hệ thống phát hiện.

- **Chi phí cập nhật cao:** Việc duy trì và cập nhật liên tục cơ sở dữ liệu chữ ký đòi hỏi nguồn lực đáng kể.

Do đó, xu hướng chuyển dịch sang **phát hiện bất thường (anomaly-based detection)** sử dụng Trí tuệ nhân tạo (AI) và Học máy (Machine Learning) đang trở thành mũi nhọn trong nghiên cứu an toàn thông tin. Thay vì tìm kiếm các mẫu tấn công cụ thể, phương pháp này học cách nhận biết hành vi "bình thường" của hệ thống và cảnh báo khi phát hiện các sai lệch đáng kể.

1.2. Từ phân loại nhị phân đến phân loại đa lớp

Hầu hết các nghiên cứu hiện nay trong lĩnh vực phát hiện xâm nhập tập trung vào bài toán **phân loại nhị phân (Binary Classification)**:

- **Lớp 0:** Bình thường (Normal) - Lưu lượng mạng hợp lệ, không chứa hành vi xâm hại.
- **Lớp 1:** Tấn công (Attack) - Lưu lượng mạng chứa hành vi độc hại.

Phương pháp này có ưu điểm đơn giản và thường đạt độ chính xác cao (như mô hình Autoencoder trong nghiên cứu này đạt 97% với AUC = 0.9861). Tuy nhiên, trong vận hành thực tế của các Trung tâm Điều hành An ninh mạng (Security Operations Center - SOC), việc chỉ biết "có tấn công" là **chưa đủ**. Quản trị viên an ninh cần biết đó là **loại tấn công gì** để có kịch bản ứng phó (Response Plan) phù hợp. Cụ thể:

- **Nếu là DoS (Denial of Service):** Cần triển khai ngay các biện pháp giảm thiểu như chặn IP nguồn tại tường lửa, kích hoạt cơ chế rate limiting, chuyển hướng lưu lượng qua CDN hoặc dịch vụ chống DDoS.
- **Nếu là Probe (Do thám):** Cần giám sát chặt chẽ các cổng và dịch vụ bị quét, rà soát cấu hình bảo mật, và có thể triển khai honeypot để thu thập thông tin về kẻ tấn công.
- **Nếu là R2L (Remote to Local):** Cần kiểm tra log đăng nhập, xác minh tính hợp lệ của các phiên làm việc từ xa, và tăng cường xác thực đa yếu tố.
- **Nếu là U2R (User to Root):** Đây là loại tấn công nguy hiểm nhất, cần **cô lập máy chủ ngay lập tức**, ngắt kết nối khỏi mạng nội bộ, kiểm tra toàn bộ log hệ thống, reset mật khẩu quản trị và tiến hành điều tra số (digital forensics).

Chính vì vậy, bài toán **Phân loại 5 lớp (5-Class Classification)** có ý nghĩa thực tiễn cao hơn trong việc hỗ trợ ra quyết định cho đội ngũ an ninh, nhưng đồng thời cũng thách thức hơn rất nhiều về mặt kỹ thuật.

1.3. Mục tiêu nghiên cứu

Báo cáo này hướng tới các mục tiêu nghiên cứu sau:

1. **Xây dựng và triển khai 7 mô hình học máy/học sâu** để phân loại lưu lượng mạng vào 5 nhóm: **Normal, DoS, Probe, R2L, U2R**, bao gồm:

- 3 mô hình học máy truyền thống: KNN, Random Forest, XGBoost phân tầng
 - 4 mô hình học sâu: LSTM, Deep SVDD, VAE, Transformer
2. **Đánh giá và so sánh hiệu quả** của các kỹ thuật xử lý dữ liệu mất cân bằng (SMOTE, Class Weighting, Hierarchical Training) khi áp dụng với từng mô hình.
 3. **Phân tích định lượng và định tính** nguyên nhân thất bại/thành công của từng mô hình đối với các lớp dữ liệu hiếm (R2L, U2R).

2. PHÂN TÍCH TẬP DỮ LIỆU NSL-KDD

2.1. Tổng quan về tập dữ liệu

Nghiên cứu sử dụng tập dữ liệu **NSL-KDD**, bản nâng cấp của tập dữ liệu KDD'99 nổi tiếng. NSL-KDD được giới thiệu vào năm 2009 bởi Tavallae và cộng sự nhằm khắc phục các vấn đề nghiêm trọng của phiên bản cũ:

- **Loại bỏ bản ghi trùng lặp:** KDD'99 chứa khoảng 78% bản ghi trùng lặp trong tập huấn luyện và 75% trong tập kiểm tra. Điều này khiến các mô hình học máy bị thiên lệch (bias) và đánh giá hiệu năng không chính xác. NSL-KDD đã loại bỏ hoàn toàn các bản ghi này.
- **Cân bằng mức độ khó:** NSL-KDD chọn lọc các bản ghi sao cho tỷ lệ các mẫu khó phân loại được cân đối hơn, giúp đánh giá khả năng tổng quát hóa của mô hình.
- **Kích thước hợp lý:** Với khoảng 126,000 mẫu huấn luyện và 22,500 mẫu kiểm tra, tập dữ liệu đủ lớn để huấn luyện các mô hình phức tạp nhưng vẫn khả thi về mặt tính toán.

Mỗi bản ghi trong tập dữ liệu đại diện cho một kết nối mạng TCP/IP và bao gồm **41 đặc trưng (features)** cùng 1 nhãn phân loại. Sau quá trình tiền xử lý với One-Hot Encoding cho các đặc trưng phân loại, số lượng đặc trưng tăng lên **122 đặc trưng**.

Thống kê tập dữ liệu sau tiền xử lý:

THÔNG SỐ	TẬP HUẤN LUYỆN	TẬP KIỂM TRA
SỐ LƯỢNG MẪU	125,973	15,660
SỐ ĐẶC TRƯNG	122	122
GIÁ TRỊ THIỂU	0	0 (sau làm sạch)

2.2. Phân tích chi tiết 5 nhóm lớp hành vi

Tập dữ liệu bao gồm 22 loại tấn công chi tiết, được gom nhóm thành 4 loại tấn công chính và 1 loại bình thường:

Normal (Bình thường):

- Các kết nối mạng hợp lệ, không chứa hành vi xâm hại. Đây là baseline để so sánh và phát hiện bất thường. Các kết nối này tuân theo các giao thức mạng chuẩn và không vi phạm chính sách bảo mật.

DoS (Denial of Service - Từ chối dịch vụ):

- Kẻ tấn công cố gắng làm quá tải tài nguyên hệ thống (CPU, RAM, băng thông mạng) khiến hệ thống không thể phục vụ người dùng hợp lệ. Đặc điểm nhận dạng: số lượng gói tin lớn trong thời gian ngắn, nhiều kết nối từ cùng nguồn, tỷ lệ lỗi cao.
- *Các loại tấn công cụ thể:* Neptune (SYN flood), Smurf (ICMP amplification), Pod (Ping of Death), Teardrop (IP fragmentation), Land (spoofed SYN), Back, Apache2, Udpstorm, Processtable, Mailbomb.

Probe (Do thám):

- Hành vi thu thập thông tin về hệ thống mục tiêu, quét cổng (port scanning), quét hệ điều hành (OS fingerprinting) để tìm lỗ hổng trước khi thực hiện tấn công chính thức. Đây thường là giai đoạn reconnaissance trong kill chain của tấn công mạng.
- *Các loại tấn công cụ thể:* Satan, Ipsweep, Nmap, Portsweep, Mscan, Saint.

R2L (Remote to Local - Tấn công từ xa vào nội bộ)

- Kẻ tấn công không có tài khoản hợp lệ trên hệ thống nhưng khai thác lỗ hổng để xâm nhập như một người dùng nội bộ. Đặc điểm: thường chỉ cần 1-2 kết nối, không tạo ra lưu lượng bất thường rõ rệt, khó phát hiện qua các đặc trưng lưu lượng.
- *Các loại tấn công cụ thể:* Guess_passwd, Ftp_write, Imap, Phf, Multihop, Warezmaster, Warezclient, Spy, Xlock, Xsnoop, Snmpguess, Snmpgetattack, Httpunnel, Sendmail, Named.

U2R (User to Root - Leo thang đặc quyền)

- Kẻ tấn công bắt đầu từ tài khoản người dùng thường (với quyền hạn hạn chế) và tìm cách chiếm quyền quản trị cao nhất (Root/Administrator). Đây là loại tấn công nguy hiểm nhất vì kẻ tấn công có thể kiểm soát hoàn toàn hệ thống sau khi thành công.
- *Các loại tấn công cụ thể:* Buffer_overflow, Loadmodule, Rootkit, Perl, Sqlattack, Xterm, Ps.

2.3. Vấn đề mất cân bằng dữ liệu nghiêm trọng

Đây là **thách thức lớn nhất** của đề tài này và cũng là vấn đề cốt lõi trong hầu hết các bài toán phát hiện xâm nhập thực tế. Phân tích thống kê chi tiết trên tập huấn luyện (KDDTrain+) cho thấy sự chênh lệch cực lớn giữa các lớp:

NHÓM LỚP	SỐ LƯỢNG MẪU	TỶ LỆ PHẦN TRĂM	TỶ LỆ SO VỚI LỚP NHỎ NHẤT	MỨC ĐỘ
NORMAL	67,343	53.46%	1,295x	Đa số (Majority)
DOS	45,927	36.46%	883x	Đa số (Majority)
PROBE	11,656	9.25%	224x	Trung bình (Minority)
R2L	995	0.79%	19x	Rất hiếm (Rare)
U2R	52	0.04%	1x	Cực hiếm (Ultra-Rare)

Phân tích tập kiểm tra (KDDTest+):

NHÓM LỚP	SỐ LƯỢNG MẪU	TỶ LỆ PHẦN TRĂM
NORMAL	8,091	51.67%
DOS	4,785	30.55%
PROBE	932	5.95%
R2L	1,824	11.65%
U2R	28	0.18%

Nhận xét quan trọng:

- Sự mất cân bằng cực độ:** Lớp U2R chỉ chiếm 0.04% dữ liệu huấn luyện (52 mẫu trên tổng số hơn 125,000 mẫu). Tỷ lệ này gần như là "nhiều thống kê" đối với các thuật toán học máy thông thường.
- Sự khác biệt phân bố giữa Train và Test:** Đáng chú ý, tỷ lệ R2L trong tập kiểm tra (11.65%) cao hơn nhiều so với tập huấn luyện (0.79%). Điều này tạo ra thách thức lớn vì mô hình học với rất ít mẫu R2L nhưng phải phân loại số lượng lớn mẫu R2L trong thực tế.
- Hệ quả với học máy:** Các thuật toán tối ưu hóa Accuracy sẽ có xu hướng coi U2R và R2L là "nhiều" và bỏ qua chúng để đạt độ chính xác tổng thể cao hơn. Một mô hình dự đoán tất cả là Normal vẫn có thể đạt Accuracy ~53%.

2.4. Phân tích đặc trưng

41 đặc trưng gốc được chia làm 3 nhóm chính, mỗi nhóm có vai trò khác nhau trong việc phát hiện các loại tấn công:

Basic Features (Đặc trưng cơ bản) - 9 đặc trưng

- Các thông tin cơ bản về kết nối TCP/IP được trích xuất trực tiếp từ header gói tin.

ĐẶC TRƯNG	MÔ TẢ	KIỂU DỮ LIỆU
DURATION	Thời gian kết nối (giây)	Số nguyên
PROTOCOL_TYPE	Giao thức (TCP, UDP, ICMP)	Phân loại

SERVICE	Dịch vụ đích (HTTP, FTP, SSH...)	Phân loại
FLAG	Trạng thái kết nối (SF, SO, REJ...)	Phân loại
SRC_BYTES	Số bytes từ nguồn đến đích	Số nguyên
DST_BYTES	Số bytes từ đích đến nguồn	Số nguyên
LAND	1 nếu nguồn = đích	Nhị phân
WRONG_FRAGMENT	Số fragment lỗi	Số nguyên
URGENT	Số gói tin urgent	Số nguyên

Hiệu quả: Nhóm này đặc biệt hữu ích cho việc phát hiện **DoS** (src_bytes cao bất thường, duration ngắn) và một số loại **Probe** (wrong_fragment).

Content Features (Đặc trưng nội dung) - 13 đặc trưng

- Các đặc trưng liên quan đến nội dung payload và hành vi đăng nhập, được trích xuất thông qua phân tích sâu gói tin.

ĐẶC TRƯNG	MÔ TẢ	Ý NGHĨA BẢO MẬT
HOT	Số chỉ báo "hot"	Hành vi đáng ngờ
NUM_FAILED_LOGINS	Số lần đăng nhập thất bại	Brute force attack
LOGGED_IN	Đăng nhập thành công hay không	Xâm nhập thành công
NUM_COMPROMISED	Số điều kiện bị xâm phạm	Mức độ nghiêm trọng
ROOT_SHELL	Có shell root hay không	Leo thang đặc quyền
SU_ATTEMPTED	Có thử lệnh su không	Leo thang đặc quyền
NUM_ROOT	Số truy cập root	Mức độ xâm nhập
NUM_FILE_CREATIONS	Số file được tạo	Hoạt động bất thường
NUM_SHELLS	Số shell prompt	Tấn công đang diễn ra
NUM_ACCESS_FILES	Số file truy cập	Đánh cắp dữ liệu
NUM_OUTBOUND_CMDS	Số lệnh outbound	Command & Control
IS_HOST_LOGIN	Đăng nhập từ host list	Xác thực
IS_GUEST_LOGIN	Đăng nhập guest	Quyền hạn thấp

Hiệu quả: Đây là nhóm đặc trưng **quan trọng nhất** để phát hiện **R2L** và **U2R**. Tuy nhiên, các giá trị này thường bằng 0 trong phần lớn các kết nối bình thường, tạo ra sự thưa thớt (sparsity) trong dữ liệu.

Traffic Features (Đặc trưng lưu lượng) - 19 đặc trưng

Các đặc trưng thống kê được tính toán trong cửa sổ thời gian 2 giây hoặc trên 100 kết nối gần nhất.

Đặc trưng theo thời gian (2 giây):

- count: Số kết nối đến cùng host
- srv_count: Số kết nối đến cùng service
- error_rate: Tỷ lệ lỗi SYN

- `srv_error_rate`: Tỷ lệ lỗi SYN theo service
- `rerror_rate`: Tỷ lệ lỗi REJ
- `same_srv_rate`: Tỷ lệ kết nối cùng service
- `diff_srv_rate`: Tỷ lệ kết nối khác service
- `srv_diff_host_rate`: Tỷ lệ kết nối đến host khác

Đặc trưng theo host (100 kết nối):

- `dst_host_count`, `dst_host_srv_count`, `dst_host_same_srv_rate`, `dst_host_diff_srv_rate`, `dst_host_same_src_port_rate`, `dst_host_srv_diff_host_rate`, `dst_host_error_rate`, `dst_host_srv_error_rate`, `dst_host_rerror_rate`, `dst_host_srv_rerror_rate`.

Hiệu quả: Nhóm này rất hiệu quả cho việc phát hiện **DoS** (count cao, `error_rate` cao) và **Probe** (`srv_count` cao với nhiều service khác nhau, `dst_host_srv_diff_host_rate` cao).

3. PHƯƠNG PHÁP NGHIÊN CỨU VÀ TIỀN XỬ LÝ

3.1. Quy trình tiền xử lý

Để chuẩn bị dữ liệu cho các mô hình học máy và học sâu, quy trình tiền xử lý sau đã được thực hiện một cách có hệ thống:

Bước 1: Làm sạch dữ liệu (Data Cleaning)

Kiểm tra giá trị Null:

- Tập huấn luyện: 0 giá trị null
- Tập kiểm tra: 3,134 mẫu có nhãn NaN → Đã loại bỏ
- Tập kiểm tra sau làm sạch: 15,660 mẫu

Việc loại bỏ các mẫu có nhãn thiếu là cần thiết vì không thể xác định ground truth cho việc đánh giá. Các mẫu bị loại được kiểm tra để đảm bảo không có sự thiên lệch (bias) theo lớp.

Bước 2: Mã hóa đặc trưng phân loại (Categorical Encoding)

Sử dụng **One-Hot Encoding** cho 3 đặc trưng định danh:

ĐẶC TRƯNG	SỐ GIÁ TRỊ GỐC	SỐ CỘT SAU OHE
PROTOCOL_TYPE	3 (TCP, UDP, ICMP)	3
SERVICE	70 (HTTP, FTP, SSH...)	70
FLAG	11 (SF, S0, REJ...)	11

Tổng số đặc trưng sau One-Hot Encoding: **122 đặc trưng** (41 gốc - 3 phân loại + 84 OHE).

Lý do chọn One-Hot Encoding: Các đặc trưng phân loại không có thứ tự tự nhiên (nominal), nên Label Encoding có thể tạo ra quan hệ thứ tự giả tạo ảnh hưởng đến mô hình.

Bước 3: Chuẩn hóa dữ liệu (Normalization)

Sử dụng **Min-Max Scaling** để đưa toàn bộ đặc trưng số về khoảng [0, 1]:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Lý do chọn Min-Max Scaling:

- Các đặc trưng như src_bytes có giá trị từ 0 đến hàng triệu bytes, trong khi các đặc trưng như logged_in chỉ là 0 hoặc 1.
- Nếu không chuẩn hóa, các đặc trưng có giá trị lớn sẽ "lấn át" (dominate) các đặc trưng nhỏ trong quá trình học.
- Min-Max Scaling phù hợp với các hàm kích hoạt sigmoid trong mạng nơ-ron.

Bước 4: Gom nhóm nhãn (Label Mapping)

Chuyển đổi 22 loại tấn công chi tiết thành 5 nhóm:

```
label_mapping = {  
    'normal': 0,  
    # DoS attacks → 1  
    'neptune': 1, 'smurf': 1, 'pod': 1, 'teardrop': 1, 'land': 1,  
    'back': 1, 'apache2': 1, 'udpstorm': 1, 'processtable': 1, 'mailbomb': 1,  
    # Probe attacks → 2  
    'satan': 2, 'ipsweep': 2, 'nmap': 2, 'portsweep': 2, 'mscan': 2, 'saint': 2,  
    # R2L attacks → 3  
    'guess_passwd': 3, 'ftp_write': 3, 'imap': 3, 'phf': 3, 'multihop': 3,  
    'warezmaster': 3, 'warezclient': 3, 'spy': 3, ...  
    # U2R attacks → 4  
    'buffer_overflow': 4, 'loadmodule': 4, 'rootkit': 4, 'perl': 4, ...  
}
```

3.2. Chiến lược xử lý mất cân bằng (Imbalance Handling)

Do tính chất "cực hiếm" của lớp R2L và U2R, nghiên cứu áp dụng và so sánh 3 chiến lược chính:

Chiến lược 1: SMOTE (Synthetic Minority Oversampling Technique)

Mô hình áp dụng: KNN, XGBoost (Phase 2)

SMOTE là kỹ thuật oversampling thông minh, thay vì chỉ sao chép lại các mẫu thiểu số (như Random Oversampling), SMOTE tạo ra các điểm dữ liệu **nhân tạo mới** bằng cách nội suy trong không gian đặc trưng.

Nguyên lý hoạt động:

1. Với mỗi mẫu thiểu số x_i , tìm k láng giềng gần nhất cùng lớp
2. Chọn ngẫu nhiên một láng giềng x_{nn}
3. Tạo mẫu mới: $x_{new} = x_i + \lambda \times (x_{nn} - x_i)$, với $\lambda \in [0,1]$

Kết quả SMOTE trên tập huấn luyện:

LỚP	TRƯỚC SMOTE	SAU SMOTE	TỶ LỆ TĂNG
NORMAL	67,343	67,343	0%
DOS	45,927	67,343	47%
PROBE	11,656	67,343	478%
R2L	995	67,343	6,668%
U2R	52	67,343	129,406%

Rủi ro của SMOTE với U2R: Khi tạo ra hơn 67,000 mẫu nhân tạo từ chỉ 52 mẫu gốc, các mẫu mới có thể không đại diện đúng cho phân bố thực tế của lớp U2R, dẫn đến overfitting trên dữ liệu giả.

Chiến lược 2: Class Weights (Trọng số lớp)

Mô hình áp dụng: Random Forest, Transformer

Thay vì thay đổi phân bố dữ liệu, phương pháp này gán **trọng số phạt** lớn hơn cho các lớp thiểu số trong hàm mất mát.

Công thức tính trọng số cân bằng: $w_c = \frac{N}{n_c \times C}$

Trong đó:

- N : Tổng số mẫu
- n_c : Số mẫu của lớp c
- C : Số lớp (5)

Trọng số tính toán được:

LỚP	SỐ MẪU	TRỌNG SỐ w_c
NORMAL	67,343	0.37
DOS	45,927	0.55
PROBE	11,656	2.16
R2L	995	25.32
U2R	52	484.49

Ý nghĩa: Nếu mô hình dự đoán sai lớp U2R, hàm mất mát sẽ bị phạt nặng gấp **~1,300 lần** so với dự đoán sai lớp Normal ($484.49/0.37 \approx 1,309$).

Chiến lược 3: Huấn luyện phân tầng (Hierarchical Training)

Mô hình áp dụng: XGBoost, Deep SVDD

Chia bài toán phân loại 5 lớp thành 2 giai đoạn dễ hơn:

Giai đoạn 1: Phân loại nhị phân Normal vs Attack

- Bài toán đơn giản hơn với 2 lớp cân bằng hơn
- Tận dụng được các mô hình có hiệu năng cao cho bài toán nhị phân (như Autoencoder)
- Mục tiêu: Recall cao để không bỏ sót tấn công

Giai đoạn 2: Phân loại chi tiết 4 loại Attack

- Chỉ áp dụng cho các mẫu được dự đoán là Attack ở giai đoạn 1
- Dữ liệu đầu vào đã được "làm giàu" tấn công, giảm bớt sự mất cân bằng
- Có thể áp dụng SMOTE riêng cho giai đoạn này

Rủi ro chính: Error Propagation - Nếu giai đoạn 1 phân loại sai một mẫu R2L/U2R thành Normal, mẫu đó sẽ bị loại bỏ và không bao giờ có cơ hội được phân loại đúng ở giai đoạn 2.

3.3. Các mô hình và cấu hình thực nghiệm

3.3.1. *K-Nearest Neighbors (KNN)*

KNN là thuật toán phi tham số (non-parametric) phân loại dựa trên khoảng cách trong không gian đặc trưng.

Cấu hình:

- Metric khoảng cách: Euclidean
- Tìm K tối ưu: Grid search từ K=1 đến K=50
- Tiêu chí chọn K: **Macro F1-score** (thay vì Accuracy để đảm bảo công bằng cho lớp hiếm)
- Kết quả: K tối ưu = **38**

- Dữ liệu: Đã qua SMOTE oversampling

Đặc điểm:

- Ưu điểm: Đơn giản, không cần huấn luyện (lazy learning)
- Nhược điểm: Chi phí tính toán cao khi dự đoán ($O(n)$ với n là số mẫu training)

3.3.2. *Random Forest (RF)*

Random Forest là ensemble của nhiều cây quyết định (Decision Trees) với kỹ thuật bagging và random feature selection.

Cấu hình:

- Số cây: `n_estimators = 100`
- Xử lý mất cân bằng: `class_weight = 'balanced'`
- Random state: 42 (đảm bảo reproducibility)
- Các tham số khác: Mặc định của scikit-learn

Đặc điểm:

- Ưu điểm: Chống overfitting tốt, tính toán song song, feature importance
- Nhược điểm: Khó giải thích (black-box), cần nhiều bộ nhớ

3.3.3. *XGBoost phân tầng (Hierarchical XGBoost)*

XGBoost (eXtreme Gradient Boosting) là thuật toán boosting mạnh mẽ với regularization.

Cấu hình Phase 1 (Normal vs Attack):

- Dữ liệu: SMOTE để cân bằng 2 lớp
- `eval_metric`: 'logloss'
- Các tham số khác: Mặc định

Cấu hình Phase 2 (4 loại Attack):

- Dữ liệu: SMOTE + Random Undersampling (Pipeline)
- `eval_metric`: 'mlogloss' (multi-class)
- Label shift: $1,2,3,4 \rightarrow 0,1,2,3$

Đặc điểm:

- Ưu điểm: Hiệu năng cao trên dữ liệu có cấu trúc, xử lý missing values

- Nhược điểm: Nhiều hyperparameters, dễ overfitting nếu không tune đúng

3.3.4. Long Short-Term Memory (LSTM)

LSTM là kiến trúc RNN đặc biệt có khả năng học các phụ thuộc dài hạn trong dữ liệu tuần tự.

Cấu hình:

Kiến trúc:

Input (sequence_length=15, features=122)

↓

LSTM(64 units, return_sequences=True, dropout=0.2)

↓

LSTM(32 units, return_sequences=False, dropout=0.2)

↓

Dense(64, ReLU) → Dropout(0.3)

↓

Dense(32, ReLU) → Dropout(0.2)

↓

Dense(5, Softmax) → Output

Hyperparameters:

- Sequence length: 15 (số mẫu liên tiếp tạo thành 1 sequence)
- Epochs: 30
- Batch size: 128
- Optimizer: Adam
- Loss: Categorical Crossentropy
- Validation split: 20%

Cách tạo sequence từ dữ liệu tabular: Mặc dù NSL-KDD là dữ liệu tabular, nghiên cứu chuyển đổi thành dạng sequence bằng cách lấy 15 bản ghi liên tiếp làm input, nhãn của sequence là nhãn của bản ghi cuối cùng. Điều này cho phép LSTM học các mẫu temporal tiềm ẩn trong luồng traffic.

3.3.5. Deep Support Vector Data Description (Deep SVDD)

Deep SVDD kết hợp neural network với One-class SVM để học hypersphere bao quanh dữ liệu bình thường.

Cấu hình:

Network Architecture:

Input (122 features)

↓

Dense(128, ReLU) → Dropout(0.1)

↓

Dense(64, ReLU) → Dropout(0.1)

↓

Dense(32, ReLU) → Dropout(0.1)

↓

Dense(16) → Representation (rep_dim=16)

Training Process:

- Phase 1: Train SVDD chỉ trên dữ liệu Normal
 - Mục tiêu: Học center μ và radius R của hypersphere
 - Loss: Khoảng cách từ representation đến center
 - Nu (ν): 0.1 (tỷ lệ outliers dự kiến)
 - Epochs: 100
- Phase 2: Train Attack Classifier
 - Input: Các mẫu được xác định là Attack ở Phase 1
 - Output: 4 lớp (DoS, Probe, R2L, U2R)
 - Epochs: 100

Ý tưởng chính:

- Normal samples nằm gần center → khoảng cách nhỏ
- Attack samples nằm xa center → khoảng cách lớn

- Threshold để phân biệt: percentile (1-v) của khoảng cách trên tập training

3.3.6. Variational Autoencoder (VAE)

VAE là generative model học latent representation của dữ liệu với ràng buộc prior distribution.

Cấu hình:

Encoder:

Input (122 features)

↓

Dense(128, ReLU) → Dropout(0.2)

↓

Dense(64, ReLU) → Dropout(0.2)

↓

Split → z_mean (32) | z_log_var (32)

↓

Sampling: $z = z_mean + \exp(0.5 * z_log_var) * \epsilon$

Decoder:

z (32 dims)

↓

Dense(64, ReLU) → Dropout(0.2)

↓

Dense(128, ReLU) → Dropout(0.2)

↓

Dense(122, Sigmoid) → Reconstruction

Classifier:

z (32 dims)

↓

Dense(64, ReLU) → Dropout(0.3)

↓

Dense(32, ReLU) $\rightarrow$ Dropout(0.2)



Dense(5, Softmax) $\rightarrow$ Classification

Loss Function: $\mathcal{L} = \mathcal{L}_{recon} + \mathcal{L}_{KL} + \mathcal{L}_{class}$

Trong đó:

- $\mathcal{L}_{recon}$: Binary Cross-Entropy giữa input và reconstruction
- $\mathcal{L}_{KL}$: KL Divergence với prior $N(0,1)$
- $\mathcal{L}_{class}$: Categorical Cross-Entropy cho classification

Hyperparameters:

- Latent dimension: 32
- Intermediate dimension: 128
- Epochs: 100
- Batch size: 128
- Loss weights: [1.0, 1.0] (reconstruction, classification)

3.3.7. *Transformer*

Transformer sử dụng cơ chế Self-Attention để học các mối quan hệ giữa các đặc trưng.

Cấu hình:

Architecture:

Input (122 features)



Feature Embedding: Linear($1 \rightarrow 128$) per feature



Positional Encoding



Transformer Encoder:

- d_model: 128
- nhead: 8 (multi-head attention)

- num_layers: 4
- dim_feedforward: 512
- dropout: 0.1

↓

Attention Pooling

↓

Classification Head:

Dense(256, ReLU) → Dropout(0.1)

↓

Dense(128, ReLU) → Dropout(0.1)

↓

Dense(5) → Output

Ý tưởng chính:

- Mỗi feature được coi như một "token"
- Self-Attention học mối quan hệ giữa các features (ví dụ: src_bytes và duration)
- Không có giả định về thứ tự (khác với LSTM)

Hyperparameters:

- Epochs: 100
- Batch size: 128
- Learning rate: 0.001
- Optimizer: Adam với weight decay 1e-4
- Scheduler: ReduceLROnPlateau (factor=0.5, patience=10)
- Class weights: Balanced

4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Tiêu chí đánh giá (Evaluation Metrics)

Trong bài toán mất cân bằng nghiêm trọng, **Accuracy (Độ chính xác tổng thể)** là chỉ số có thể gây hiểu nhầm. Ví dụ: một mô hình dự đoán tất cả là Normal vẫn có thể đạt Accuracy > 50%. Do đó, báo cáo sử dụng các chỉ số chi tiết sau:

Precision (Độ chính xác dự báo):

$$\text{Precision} = \frac{TP}{TP + FP}$$

Tỷ lệ các dự đoán dương thực sự đúng. Quan trọng khi chi phí False Positive cao (báo động giả gây tốn nguồn lực điều tra).

Recall (Độ phủ / Độ nhạy):

$$\text{Recall} = \frac{TP}{TP + FN}$$

Tỷ lệ các mẫu dương thực tế được phát hiện. Quan trọng khi chi phí False Negative cao (bỏ sót tấn công nguy hiểm).

F1-Score:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Trung bình điều hòa của Precision và Recall, cân bằng giữa hai chỉ số.

Macro Avg F1:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C F1_c$$

Trung bình cộng F1 của tất cả các lớp, **mỗi lớp có trọng số bằng nhau**. Đây là chỉ số quan trọng nhất để đánh giá công bằng hiệu năng trên lớp hiếm.

Weighted Avg F1

$$\text{Weighted-F1} = \sum_{c=1}^C \frac{n_c}{N} \times F1_c$$

Trung bình có trọng số theo số lượng mẫu của mỗi lớp. Phản ánh hiệu năng tổng thể nhưng thiên về lớp đa số.

4.2. Kết quả mô hình K-Nearest Neighbors (KNN)

Mô hình KNN được huấn luyện trên dữ liệu đã qua SMOTE với K tối ưu = 38 (tìm bằng grid search dựa trên Macro F1).

LỚP	PRECISION	RECALL	F1-SCORE	SUPPORT
NORMAL (0)	0.52	0.57	0.54	8,091
DOS (1)	0.30	0.29	0.30	4,785
PROBE (2)	0.06	0.10	0.08	932
R2L (3)	0.09	0.03	0.04	1,824
U2R (4)	0.00	0.00	0.00	28
ACCURACY			0.39	15,660
MACRO AVG	0.19	0.20	0.19	
WEIGHTED AVG	0.36	0.39	0.37	

Phân tích kết quả:

- **Hiệu năng tổng thể rất thấp:** Accuracy chỉ đạt 39%, thấp hơn cả việc đoán ngẫu nhiên trong một số trường hợp.
- **Lớp U2R hoàn toàn không nhận diện được:** F1 = 0.00, mô hình không dự đoán đúng bất kỳ mẫu U2R nào.
- **Lớp R2L gần như thất bại:** Recall chỉ 0.03, nghĩa là chỉ 3% mẫu R2L thực tế được phát hiện.
- **Nguyên nhân sâu xa:**
 1. KNN dựa trên khoảng cách Euclidean trong không gian 122 chiều, bị ảnh hưởng bởi "curse of dimensionality"
 2. Các mẫu SMOTE nhân tạo có thể đã tạo ra vùng overlap giữa các lớp, làm mờ ranh giới quyết định
 3. Với K=38, quyết định dựa trên đa số trong 38 láng giềng, dễ bị áp đảo bởi các lớp đa số ngay cả sau SMOTE

4.3. Kết quả mô hình Random Forest (RF)

Mô hình sử dụng trọng số lớp cân bằng (class_weight='balanced') để xử lý mất cân bằng.

LỚP	PRECISION	RECALL	F1-SCORE	SUPPORT
NORMAL (0)	0.52	0.65	0.58	8,091
DOS (1)	0.30	0.30	0.30	4,785
PROBE (2)	0.07	0.05	0.06	932
R2L (3)	0.07	0.00	0.00	1,824
U2R (4)	0.00	0.00	0.00	28
ACCURACY			0.43	15,660

MACRO AVG	0.19	0.20	0.19
WEIGHTED AVG	0.35	0.43	0.38

Phân tích kết quả:

- **Cải thiện nhẹ so với KNN:** Accuracy tăng từ 39% lên 43%, chủ yếu nhờ cải thiện nhận diện lớp Normal (Recall 0.65 so với 0.57).
- **R2L và U2R hoàn toàn thất bại:** F1 = 0.00 cho cả hai lớp. Đặc biệt R2L có Recall = 0.00, nghĩa là không phát hiện được bất kỳ mẫu nào trong 1,824 mẫu R2L.
- **Nguyên nhân:**
 1. Class weights tăng penalty nhưng không đủ để thay đổi splitting criteria của Decision Trees
 2. Các đặc trưng phân biệt R2L và U2R (Content features) có giá trị sparse (phần lớn = 0)
 3. Random Forest ưu tiên các splits dựa trên đặc trưng có information gain cao, thường là Traffic features (hiệu quả cho DoS/Probe nhưng không cho R2L/U2R)

4.4. Kết quả mô hình XGBoost (Chiến lược phân tầng)

Mô hình sử dụng chiến lược 2 giai đoạn: Phase 1 phân loại Normal/Attack, Phase 2 phân loại chi tiết 4 loại Attack.

Kết quả Phase 1 (Normal vs Attack):

- Số mẫu được dự đoán là Attack và chuyển sang Phase 2: ~9,500 mẫu
- Một số mẫu R2L/U2R bị phân loại sai thành Normal và bị loại

LỚP	PRECISION	RECALL	F1-SCORE	SUPPORT
NORMAL (0)	0.52	0.61	0.56	8,091
DOS (1)	0.30	0.30	0.30	4,785
PROBE (2)	0.07	0.07	0.07	932
R2L (3)	0.09	0.02	0.03	1,824
U2R (4)	0.00	0.00	0.00	28
ACCURACY			0.42	15,660
MACRO AVG	0.20	0.20	0.19	
WEIGHTED AVG	0.36	0.42	0.38	

Phân tích kết quả:

- **Không có cải thiện đáng kể:** Accuracy 42% tương đương Random Forest.
- **Vấn đề Error Propagation nghiêm trọng:**

1. Nhiều mẫu R2L và U2R có đặc điểm traffic "bình thường" (ít gói tin, không tăng đột biến bandwidth)
 2. Phase 1 phân loại nhằm chúng thành Normal, loại bỏ khỏi Phase 2
 3. Recall của R2L chỉ 0.02 (2%), nghĩa là 98% mẫu R2L bị bỏ sót
- **Hierarchical approach không giải quyết được vấn đề cốt lõi:** Sự khó khăn nằm ở việc các đặc trưng hiện tại không đủ để phân biệt R2L/U2R với Normal, không phải do cấu trúc bài toán.

4.5. Kết quả mô hình LSTM

LSTM được thiết kế để học các mẫu temporal trong luồng traffic bằng cách chuyển đổi dữ liệu tabular thành sequences.

LỚP	PRECISION	RECALL	F1-SCORE	SUPPORT
NORMAL (0)	0.75	0.82	0.78	8,091
DOS (1)	0.68	0.71	0.69	4,785
PROBE (2)	0.45	0.38	0.41	932
R2L (3)	0.32	0.18	0.23	1,824
U2R (4)	0.15	0.07	0.10	28
ACCURACY			0.765	15,660
MACRO AVG	0.47	0.43	0.44	
WEIGHTED AVG	0.63	0.765	0.71	

Phân tích kết quả:

- **Cải thiện vượt bậc so với ML truyền thống:** Accuracy tăng từ 43% lên **76.5%**, gần gấp đôi.
- **Bước đột phá với lớp hiếm:**
 - R2L: F1 tăng từ 0.03 lên **0.23** (tăng ~7.7 lần)
 - U2R: F1 tăng từ 0.00 lên **0.10** (từ không nhận diện được đến nhận diện được một phần)
- **Nguyên nhân thành công:**
 - **Sequence modeling:** Việc xem 15 mẫu liên tiếp như một sequence cho phép LSTM học được các mẫu temporal mà các mô hình tabular bỏ qua
 - **Non-linear transformation:** Các lớp LSTM và Dense học được các biến đổi phi tuyến phức tạp của đặc trưng
 - **Dropout regularization:** Giúp giảm overfitting trên các lớp đa số
- **Hạn chế còn tồn tại:**

- Recall của U2R chỉ 7% (2/28 mẫu), vẫn bỏ sót phần lớn tấn công nguy hiểm này
- Validation accuracy trong training đạt >99% nhưng test accuracy 76.5% → Có dấu hiệu overfitting

4.6. Kết quả mô hình Deep SVDD

Deep SVDD sử dụng phương pháp one-class learning cho Phase 1, sau đó phân loại chi tiết ở Phase 2.

LỚP	PRECISION	RECALL	F1-SCORE	SUPPORT
NORMAL (0)	0.72	0.78	0.75	8,091
DOS (1)	0.65	0.68	0.66	4,785
PROBE (2)	0.42	0.35	0.38	932
R2L (3)	0.28	0.15	0.20	1,824
U2R (4)	0.12	0.04	0.06	28
ACCURACY			0.72	15,660
MACRO AVG	0.44	0.40	0.41	
WEIGHTED AVG	0.60	0.72	0.66	

Phân tích kết quả:

- **Hiệu năng tốt nhưng thấp hơn LSTM:** Accuracy 72% so với 76.5% của LSTM.
- **Ưu điểm của phương pháp one-class:**
 1. Phase 1 học "bình thường là gì" mà không cần nhãn tấn công
 2. Có khả năng phát hiện các tấn công mới (zero-day) nằm ngoài hypersphere
- **Hạn chế:**
 1. Một số tấn công R2L/U2R có representation gần với Normal, nằm trong hypersphere
 2. Phase 2 vẫn gặp vấn đề mất cân bằng khi phân loại 4 loại attack

4.7. Kết quả mô hình VAE

VAE kết hợp unsupervised learning (reconstruction) với supervised learning (classification).

LỚP	PRECISION	RECALL	F1-SCORE	SUPPORT
NORMAL (0)	0.73	0.79	0.76	8,091
DOS (1)	0.66	0.69	0.67	4,785
PROBE (2)	0.43	0.36	0.39	932
R2L (3)	0.29	0.16	0.21	1,824
U2R (4)	0.14	0.04	0.06	28
ACCURACY			0.73	15,660
MACRO AVG	0.45	0.41	0.42	

WEIGHTED AVG	0.61	0.73	0.67
-----------------	------	------	-------------

Phân tích kết quả:

- **Hiệu năng tương đương Deep SVDD:** Accuracy 73%, Macro F1 0.42.
- **Ý nghĩa của latent space:**
 1. VAE học latent representation 32 chiều, nén thông tin từ 122 đặc trưng
 2. Các mẫu cùng lớp có xu hướng cluster gần nhau trong latent space
 3. Tuy nhiên, overlap giữa các cluster vẫn đáng kể, đặc biệt R2L/Normal
- **Lợi ích của hybrid approach:**
 1. Reconstruction loss giúp model học features quan trọng
 2. Classification loss hướng dẫn model tập trung vào features discriminative

4.8. Kết quả mô hình Transformer

Transformer sử dụng self-attention để học mối quan hệ giữa các đặc trưng mà không cần giả định tuần tự.

LỚP	PRECISION	RECALL	F1-SCORE	SUPPORT
NORMAL (0)	0.78	0.84	0.81	8,091
DOS (1)	0.71	0.74	0.72	4,785
PROBE (2)	0.48	0.42	0.45	932
R2L (3)	0.35	0.21	0.26	1,824
U2R (4)	0.18	0.11	0.14	28
ACCURACY			0.782	15,660
MACRO AVG	0.50	0.46	0.48	
WEIGHTED AVG	0.66	0.782	0.73	

Phân tích kết quả:

- **Hiệu năng tốt nhất trong tất cả các mô hình:** Accuracy **78.2%**, Macro F1 **0.48**.
- **Cải thiện đáng kể cho lớp hiếm:**
 - R2L: F1 = **0.26** (cao nhất trong tất cả models)
 - U2R: F1 = **0.14** (cao nhất trong tất cả models)
 - Recall U2R = 11% (3/28 mẫu) - vẫn thấp nhưng là kết quả tốt nhất
- **Nguyên nhân thành công của Transformer:**
 1. **Self-Attention mechanism:** Học được mối quan hệ giữa tất cả các cặp features, ví dụ: (num_failed_logins, logged_in) quan trọng cho R2L

2. **Global context:** Không như LSTM chỉ xem tuần tự, Transformer xem toàn bộ features cùng lúc

3. **Class weights + Deep architecture:** Kết hợp hiệu quả

- **Visualization attention weights** (hướng nghiên cứu tiếp theo):
 - Có thể trực quan hóa features nào được model chú ý nhất cho từng lớp
 - Giúp giải thích quyết định của model (Explainability)

5. THẢO LUẬN VÀ SO SÁNH

5.1. Tổng hợp kết quả và so sánh mô hình

Bảng tổng hợp kết quả của 7 mô hình trên tập kiểm tra NSL-KDD:

Mô hình	Accuracy	Macro F1	F1 (Normal)	F1 (DoS)	F1 (Probe)	F1 (R2L)	F1 (U2R)
KNN (SMOTE)	39.0%	0.19	0.54	0.30	0.08	0.04	0.00
Random Forest	43.0%	0.19	0.58	0.30	0.06	0.00	0.00
XGBoost (Hier.)	42.0%	0.19	0.56	0.30	0.07	0.03	0.00
LSTM	76.5%	0.44	0.78	0.69	0.41	0.23	0.10
Deep SVDD	72.0%	0.41	0.75	0.66	0.38	0.20	0.06
VAE	73.0%	0.42	0.76	0.67	0.39	0.21	0.06
Transformer	78.2%	0.48	0.81	0.72	0.45	0.26	0.14

Nhận xét chung:

1. **Khoảng cách hiệu năng lớn giữa ML truyền thống và Deep Learning:**

- ML (KNN, RF, XGBoost): Accuracy 39-43%, Macro F1 ~0.19
- DL (LSTM, SVDD, VAE, Transformer): Accuracy 72-78%, Macro F1 0.41-0.48
- Chênh lệch: ~35 điểm phần trăm Accuracy, ~0.25 điểm Macro F1

2. **Transformer đạt hiệu năng tốt nhất** trên hầu hết các chỉ số, đặc biệt là F1 của các lớp hiếm.

3. **Sự cải thiện với lớp hiếm vẫn còn hạn chế:**

- U2R tốt nhất: F1 = 0.14 (Transformer) → vẫn bỏ sót 89% tấn công
- R2L tốt nhất: F1 = 0.26 (Transformer) → vẫn bỏ sót 79% tấn công

5.2. Phân tích sự thất bại trong nhận diện lớp hiếm

Nguyên nhân 1: Sự "vô hình" về mặt thống kê

Với chỉ 52 mẫu U2R trong tập huấn luyện (125,973 mẫu), các mô hình học máy thống kê không có đủ dữ liệu để:

- Học được phân bố đặc trưng của lớp U2R
- Khái quát hóa cho các mẫu U2R mới trong tập kiểm tra (28 mẫu, có thể khác biệt với 52 mẫu training)

Minh họa bằng số:

- Nếu mô hình học K=10 láng giềng, xác suất có ít nhất 1 láng giềng U2R trong vùng lân cận rất thấp
- Decision Trees cần ít nhất vài mẫu để tạo split, khó có split cho U2R

Nguyên nhân 2: Feature Overlap (Chồng lấn đặc trưng)

Phân tích correlation giữa các đặc trưng và nhãn cho thấy:

ĐẶC TRƯNG	CORRELATION VỚI DOS	CORRELATION VỚI R2L	CORRELATION VỚI U2R
SRC_BYTES	0.42	0.05	0.03
COUNT	0.51	-0.08	-0.02
SERROR_RATE	0.38	0.02	0.01
NUM_FAILED_LOGINS	-0.02	0.12	0.08
ROOT_SHELL	-0.01	0.01	0.15
LOGGED_IN	0.05	0.18	0.12

Nhận xét:

- **DoS và Probe** có correlation cao với Traffic features (count, error_rate) → Dễ phát hiện
- **R2L và U2R** có correlation yếu với hầu hết các features → Khó phân biệt với Normal
- Content features (num_failed_logins, root_shell) quan trọng cho R2L/U2R nhưng có giá trị sparse

Nguyên nhân 3: Bản chất tấn công khác biệt

LOẠI TẤN CÔNG	HÀNH VI ĐẶC TRƯNG	DẤU HIỆU TRONG TRAFFIC
DOS	Gửi hàng nghìn gói tin/giây	count cao, src_bytes lớn, duration ngắn
PROBE	Quét nhiều port/host	srv_count cao, dst_host_diff srv_rate cao
R2L	Khai thác lỗ hổng qua 1-2 kết nối	Gần như không khác Normal về traffic

U2R

Leo thang quyền local

Hành vi xảy ra sau khi đã logged_in, khó phát hiện từ network traffic

Vấn đề cốt lõi: R2L và U2R là các tấn công "giấu mình", không tạo ra bất thường rõ ràng trong lưu lượng mạng. Để phát hiện chúng hiệu quả, cần:

- Phân tích nội dung gói tin (deep packet inspection)
- Kết hợp với log hệ thống (system call, authentication logs)
- Sử dụng behavioral analytics dài hạn

5.3. So sánh với hiệu quả của bài toán nhị phân

Để thấy rõ sự sụt giảm hiệu năng khi chuyển từ phát hiện (Detection) sang định danh (Identification), ta so sánh với kết quả bài toán Nhị phân:

BÀI TOÁN	MÔ HÌNH	ACCURACY	MACRO F1	AUC	KHẢ NĂNG
NHỊ PHÂN	Autoencoder	97%	0.97	0.9861	Xuất sắc
NHỊ PHÂN	Random Forest	89%	0.88	0.92	Rất tốt
5 LỚP	Transformer	78%	0.48	-	Khá
5 LỚP	LSTM	76.5%	0.44	-	Khá
5 LỚP	Random Forest	43%	0.19	-	Kém

Kết luận quan trọng:

1. **Sự sụt giảm hiệu năng là đáng kể nhưng tất yếu:**
 - Accuracy giảm từ 97% (nhị phân) xuống 78% (5 lớp) với Deep Learning
 - Điều này phản ánh độ khó thực sự của bài toán phân loại chi tiết
2. **Autoencoder hiệu quả cho nhị phân nhưng không áp dụng được cho đa lớp:**
 - Autoencoder học "bình thường là gì" và coi mọi bất thường là tấn công
 - Không có khả năng phân biệt DoS với U2R - cả hai đều "bất thường"
3. **Trade-off giữa Phát hiện và Định danh:**
 - Trong thực tế, có thể xây dựng hệ thống 2 tầng:
 - Tầng 1: Autoencoder phát hiện tấn công với độ chính xác cao
 - Tầng 2: Transformer phân loại chi tiết các mẫu bị đánh dấu là tấn công

5.4. Phân tích ưu nhược điểm của từng phương pháp

Mô hình ML truyền thống (KNN, RF, XGBoost)

Ưu điểm	Nhược điểm
---------	------------

Huấn luyện nhanh (phút)	Hiệu năng thấp với dữ liệu phức tạp
Dễ giải thích (Feature Importance)	Không học được non-linear interactions
Ít yêu cầu tài nguyên	Nhạy cảm với class imbalance
Không cần GPU	Không tổng quát hóa tốt

Mô hình Deep Learning (LSTM, Transformer, VAE, SVDD)

Ưu điểm	Nhược điểm
Hiệu năng cao với dữ liệu phức tạp	Huấn luyện chậm (giờ)
Học được non-linear patterns	Cần nhiều dữ liệu
Khả năng tổng quát hóa tốt hơn	Khó giải thích (black-box)
Transfer learning potential	Yêu cầu GPU cho training

6. KẾT LUẬN

Nghiên cứu này đã thực hiện đánh giá toàn diện 7 mô hình học máy và học sâu trên bài toán phân loại 5 lớp hành vi mạng sử dụng tập dữ liệu NSL-KDD. Các kết luận chính bao gồm:

1. Về hiệu năng các mô hình

- **Transformer** đạt hiệu năng tốt nhất với Accuracy **78.2%** và Macro F1 **0.48**, vượt trội so với tất cả các mô hình khác.
- **LSTM** đứng thứ hai với Accuracy **76.5%**, chứng minh khả năng của sequence modeling cho dữ liệu mạng.
- Các mô hình học máy truyền thống (KNN, RF, XGBoost) đạt hiệu năng thấp (Accuracy 39-43%), cho thấy giới hạn của các phương pháp này với dữ liệu mất cân bằng nghiêm trọng.

2. Về thách thức phân loại lớp hiếm

- **Lớp U2R** (0.04% dữ liệu) vẫn là thách thức lớn nhất. Ngay cả mô hình tốt nhất (Transformer) cũng chỉ đạt $F1 = 0.14$, bỏ sót 89% tấn công leo thang đặc quyền.
- **Lớp R2L** có cải thiện đáng kể với Deep Learning (F1 từ 0.00 lên 0.26) nhưng vẫn chưa đủ tin cậy cho triển khai thực tế.
- Nguyên nhân chính là bản chất "ẩn mình" của R2L và U2R - chúng không tạo ra bất thường rõ ràng trong lưu lượng mạng.

3. Về các kỹ thuật xử lý mất cân bằng

- **SMOTE** tạo ra quá nhiều mẫu nhân tạo từ ít mẫu gốc, có thể gây overfitting và làm mờ ranh giới lớp.

- **Class Weights** hiệu quả hơn với Deep Learning nhưng không đủ mạnh cho mô hình ML truyền thống.
- **Hierarchical Training** gặp vấn đề Error Propagation, đặc biệt nghiêm trọng với các lớp có đặc trưng giống Normal.

4. Về ý nghĩa thực tiễn

- Việc phân loại chi tiết 5 lớp **khó hơn đáng kể** so với phát hiện nhị phân (Accuracy giảm từ 97% xuống 78%).
- Với hiệu năng hiện tại, hệ thống có thể hỗ trợ analyst SOC nhưng **không nên tự động hóa hoàn toàn** quyết định ứng phó.
- Cần kết hợp với các nguồn dữ liệu khác (log hệ thống, endpoint telemetry) để phát hiện hiệu quả R2L và U2R.